



Johannes Segitz

Security for
(not only **openSUSE**) developers

Whoami

Johannes Segitz (@swars@infosec.exchange)

Security engineer at SUSE (Nuremberg, Germany)

- Auditing/code review (not that much nowadays)
- Product pentesting
- SELinux expert



Whoami

Johannes Segitz (@swars@infosec.exchange)

Security engineer at SUSE (Nuremberg, Germany)

- Auditing/code review (not that much nowadays)
- Product pentesting
- SELinux expert

Yogyakarta is great (but a bit too hot), thanks for having me here!



Outline

We will talk about:

- Quick introduction to the SUSE product security team



Outline

We will talk about:

- Quick introduction to the SUSE product security team
- Some interesting ways to hack (not only) openSUSE developers



Outline

We will talk about:

- Quick introduction to the SUSE product security team
- Some interesting ways to hack (not only) openSUSE developers
- Some general security recommendations



The SUSE product security team

Important: **Product** security for SUSE, there are other security teams nowadays



The SUSE product security team

Important: **Product** security for SUSE, there are other security teams nowadays

Split up into two groups:

- Reactive security: Mainly incident handling: For SUSE and openSUSE



The SUSE product security team

Important: **Product** security for SUSE, there are other security teams nowadays

Split up into two groups:

- Reactive security: Mainly incident handling: For SUSE and openSUSE
- Proactive security: Security work before shipment, e.g. hardening



The SUSE product security team

Important: **Product** security for SUSE, there are other security teams nowadays

Split up into two groups:

- Reactive security: Mainly incident handling: For SUSE and openSUSE
- Proactive security: Security work before shipment, e.g. hardening

Meaningful work, doing a lot of our proactive work on openSUSE



Security for developers

Trust all the way down

Supply chain attacks are becoming more prominent all the time



Security for developers

Trust all the way down

Supply chain attacks are becoming more prominent all the time

The baddies are pretty professional nowadays (or leverage AI for scale)

Attacking early in the chain is a good way for them to maximize their impact



Security for developers

Trust all the way down

You and your workstations are part of the *root of trust*

Developer workstation compromised → very hard to mitigate later in the chain



Security for developers

Trust all the way down

You and your workstations are part of the *root of trust*

Developer workstation compromised → very hard to mitigate later in the chain

Example: Developer of popular noise suppression tool NoiseTorch has dev machine compromised

<https://www.gamingonlinux.com/2022/05/developer-of-popular-noise-suppression-tool-noisetorch-has-dev-machine-compromised/>



Security for developers

XZ Outbreak (CVE-2024-3094)



XZ Utils is a collection of open-source tools and libraries for the XZ compression format, that are used for high compression ratios with support for multiple compression algorithms, notably LZMA2.



On Friday 29th of March, Andres Freund (principal software engineer at Microsoft) emailed oss-security informing the community of the discovery of a backdoor in xz/liblzma version 5.6.0 and 5.6.1.



Github Activity Summary (user: JiaT75)



Repository:
<https://github.com/tukaani-project/xz>

JiaT75's **first commit**
to the XZ repo

2022-02-06

PR opened in oss-fuzz to
disable ifunc for fuzzing
builds. Allegedly to mask the
malicious changes.

2023-07-08

Obfuscated/encrypted stages binary backdoor
hidden in two test files:

- `tests/files/bad-3-corrupt_lzma2.xz`
- `tests/files/good-large_compressed.lzma`

2024-03-09



2021

User **Jia Tan (JiaT75)**
creates his Github Account

2023-06-28

Potential infrastructure testing:
liblzma: "Add ifunc implementation
to crc64_fast.c."

2024-02-16

Malicious "`build-to-host.m4`" file added
to .gitignore, later incorporated to the
package release.



xz/liblzma
v5.6.0 & v5.6.1

Graphic by Thomas Roccia (@fr0gger@infosec.exchange)



Security for developers

Assumptions about the attackers (slide from 2022 ...)

Distributions are high value targets



Security for developers

Assumptions about the attackers (slide from 2022 ...)

Distributions are high value targets

Assumptions

- organized group
- capable, equipped and (maybe state) funded
- performing long term attacks



Security for developers

Assumptions about the attackers (slide from 2022 ...)

Distributions are high value targets

Assumptions

- organized group
- capable, equipped and (maybe state) funded
- performing long term attacks

Not showing high end attacks here, they're not necessary

This is the attacker you should assume to be up against



Security for developers

Gameshow voice: Will this compromise your workstation?

```
git checkout https://github.com/foo/bar  
cd bar  
make
```

Show of hands: Is your account compromised?



Security for developers

Gameshow voice: Will this compromise your workstation?

```
git checkout https://github.com/foo/bar  
cd bar  
make
```

Show of hands: Is your account compromised?

Yes, very easily. Immediate command execution



Security for developers

Gameshow voice: Will this compromise your workstation?

```
git checkout https://github.com/foo/bar  
cd bar  
cd subdir  
git status
```

Show of hands: Is your account compromised?



Security for developers

Gameshow voice: Will this compromise your workstation?

```
git checkout https://github.com/foo/bar  
cd bar  
cd subdir  
git status
```

Show of hands: Is your account compromised?

Yes, via malicious embedded bare repositories



Security for developers

Gameshow voice: Will this compromise your workstation?

```
osc checkout home:jsegitz:branches:presentation_demo vim  
cd vim  
quilt setup vim.spec
```

Show of hands: Is your account compromised?



Security for developers

Gameshow voice: Will this compromise your workstation?

```
osc checkout home:jsegitz:branches:presentation_demo vim  
cd vim  
quilt setup vim.spec
```

Show of hands: Is your account compromised?

Yes. Quilt is executing commands from the spec file directly



Security for developers

Gameshow voice: Will this compromise your workstation?

```
cd home:jsegitz:security_for_developers/demo4  
osc build
```

Show of hands: Is your account compromised?



Security for developers

Gameshow voice: Will this compromise your workstation?

```
cd home:jsegitz:security_for_developers/demo4  
osc build
```

Show of hands: Is your account compromised?

Depends. (Default) chroot build + obs-service-replace_using_env
service installed → **code execution as root on your system**



Security for developers

Gameshow voice: Will this compromise your workstation?

So now you switched to KVM building because this is secure, right?

```
osc co home:jsegitz:security_for_developers/demo5  
cd home:jsegitz:security_for_developers/demo5  
osc build --vm-type=kvm
```

Show of hands: Is your account compromised?



Security for developers

Gameshow voice: Will this compromise your workstation?

So now you switched to KVM building because this is secure, right?

```
osc co home:jsegitz:security_for_developers/demo5  
cd home:jsegitz:security_for_developers/demo5  
osc build --vm-type=kvm
```

Show of hands: Is your account compromised?

yes, code execution as your local user **on the host** if
`obs-service-replace_using_env` service is available



Security for developers

Gameshow voice: Will this compromise your workstation?

Lets jump to the way many people work now. You're AI buddy is doing the heavy lifting

```
agy --dangerously-skip-permissions
```

```
Accessing workspace:
```

```
/root/src
```

```
Do you trust the contents of this project?
```

```
Antigravity CLI requires permission to read, edit, and execute files here.
```

```
> Yes, I trust this folder
```

```
No, exit
```

```
↑/↓ Navigate · enter Confirm
```

Show of hands: Is your account compromised?



Takeaway

Many operations you do daily carry more risk than people expect



Takeaway

Many operations you do daily carry more risk than people expect
In my opinion it doesn't make sense to identify risky actions



Recommendations

High level

General recommendations:

- Apply updates regularly, optimally automate this



Recommendations

High level

General recommendations:

- Apply updates regularly, optimally automate this
- Use (strong) 2FA



Recommendations

High level

General recommendations:

- Apply updates regularly, optimally automate this
- Use (strong) 2FA
- Use password manager, split up DBs



Recommendations

High level

General recommendations:

- Apply updates regularly, optimally automate this
- Use (strong) 2FA
- Use password manager, split up DBs
- Use different browsers for different concerns
 - with adblock (uBlock Origin, pi-hole)
 - a way to block Javascript (uMatrix)



Recommendations

High level

In general: As much separation and least privilege as possible

It's convinient to do all your work with one account, I know



Recommendations

High level

In general: As much separation and least privilege as possible

It's convinient to do all your work with one account, I know

Security is most often a matter of mindset



Recommendations

Mid level

Create an environment that is distinct from your main environment

- Different Linux user
- containers / namespacing tools
- VMs

Automate this, work in there



Recommendations

Mid level

Create an environment that is distinct from your main environment

- Different Linux user
- containers / namespacing tools
- VMs

Automate this, work in there

Simpler to use → better



Recommendations

Mid level

Create an environment that is distinct from your main environment

- Different Linux user
- containers / namespacing tools
- VMs

Automate this, work in there

Simpler to use → better

Prevent persistence



Recommendations

Low level for AI usage

GLAIPNIR Project: <https://github.com/val4oss/ai-agents-sandbox/>

Secure environment for AI coding agents

- rootless container with Podman
- microvm with libkrun



Recommendations

Low level for AI usage

GLAIPNIR Project: <https://github.com/val4oss/ai-agents-sandbox/>

Secure environment for AI coding agents

- rootless container with Podman
- microvm with libkrun

Agents showed that they can break out of (some) sandboxes, so be aware of that



Recommendations

Low level for OBS

Harden your osc configuration. Use VMs for building

```
# build-type to use
# empty      -  chroot build
# kvm        -  kvm VM build
# (needs build-device, build-swap, build-memory)
# xen        -  xen VM build
# (needs build-device, build-swap, build-memory)
#   experimental:
#       qemu  -  qemu VM build
#       lxc   -  lxc build
build-type = kvm
```



Isolation is not a magic bullet

You need to think like an attacker



Isolation is not a magic bullet

You need to think like an attacker

You sandbox me during development?

Great, what about when this is running later on?



Isolation is not a magic bullet

You need to think like an attacker

You sandbox me during development?

Great, what about when this is running later on?

Developing in a sandbox doesn't make the output safe



Thanks

Thank you for your attention!

If you have question:

- Ask me now :)
- I'll be available for questions during the conference
- Later on reach out to the security team via security@suse.de



Some sweet

Come and grab some Lebkuchen and Spekulatius



Some sweet

Come and grab some Lebkuchen and Spekulatius



Everything is global now :)



Why this talk?

Many people rely on wrong threat models

